

```

import pandas as pd
import numpy as np

np.random.seed(42)

n = 1000

df = pd.DataFrame({
    "Age": np.random.randint(18, 70, n),
    "MonthlyCharges": np.random.randint(20, 150, n),
    "Tenure": np.random.randint(1, 72, n),
    "ContractType": np.random.choice(
        ["Month-to-Month", "One Year", "Two Year"],
        n
    ),
    "InternetService": np.random.choice(
        ["DSL", "Fiber", "None"],
        n
    ),
    "CustomerSupportCalls": np.random.randint(0, 10, n)
})

churn_prob = (
    (df["ContractType"] == "Month-to-Month") * 0.3 +
    (df["MonthlyCharges"] > 100) * 0.2 +
    (df["Tenure"] < 12) * 0.3 +
    (df["CustomerSupportCalls"] > 5) * 0.2
)

df["Churn"] = np.where(
    np.random.rand(n) < churn_prob,
    "Yes",
    "No"
)

df.to_csv(
    "Customer_Churn.csv",
    index=False
)

print(df.head())
print(df.shape)

```

	Age	MonthlyCharges	Tenure	ContractType	InternetService	\
0	56	134	36	Month-to-Month	DSL	
1	69	34	36	One Year	DSL	
2	46	138	3	One Year	Fiber	
3	32	47	8	One Year	DSL	
4	60	58	70	Month-to-Month	DSL	

	CustomerSupportCalls	Churn
0	5	No
1	5	No
2	2	No
3	7	No
4	0	No

(1000, 7)

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

from sklearn.tree import (
    DecisionTreeClassifier,
    plot_tree
)

```

```
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    ConfusionMatrixDisplay
)
```

```
df = pd.read_csv(
    "Customer_Churn.csv"
)

df.head()
```

	Age	MonthlyCharges	Tenure	ContractType	InternetService	CustomerSupportCalls	Churn	
0	56	134	36	Month-to-Month	DSL	5	No	
1	69	34	36	One Year	DSL	5	No	
2	46	138	3	One Year	Fiber	2	No	
3	32	47	8	One Year	DSL	7	No	
4	60	58	70	Month-to-Month	DSL	0	No	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
print(df.shape)

df.info()

df.describe(include='all')
```

```
(1000, 7)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Age                    1000 non-null   int64
1   MonthlyCharges         1000 non-null   int64
2   Tenure                  1000 non-null   int64
3   ContractType            1000 non-null   object
4   InternetService         658 non-null    object
5   CustomerSupportCalls    1000 non-null   int64
6   Churn                   1000 non-null   object
dtypes: int64(4), object(3)
memory usage: 54.8+ KB
```

	Age	MonthlyCharges	Tenure	ContractType	InternetService	CustomerSupportCalls	Churn
count	1000.00000	1000.000000	1000.000000	1000	658	1000.0000	1000
unique	NaN	NaN	NaN	3	2	NaN	NaN
top	NaN	NaN	NaN	Month-to-Month	DSL	NaN	NaN
freq	NaN	NaN	NaN	345	332	NaN	NaN
mean	43.81900	83.095000	35.127000	NaN	NaN	4.6080	NaN
std	14.99103	37.910621	20.342386	NaN	NaN	2.8214	NaN
min	18.00000	20.000000	1.000000	NaN	NaN	0.0000	NaN
25%	31.00000	51.000000	18.000000	NaN	NaN	2.0000	NaN
50%	44.00000	82.000000	35.000000	NaN	NaN	5.0000	NaN
75%	56.00000	117.000000	52.000000	NaN	NaN	7.0000	NaN
max	69.00000	149.000000	71.000000	NaN	NaN	9.0000	NaN

```
print(df.isnull().sum())
```

```
Age                    0
MonthlyCharges         0
Tenure                  0
ContractType            0
InternetService        342
CustomerSupportCalls    0
Churn                   0
dtype: int64
```

```
le = LabelEncoder()

for col in df.columns:

    if df[col].dtype == "object":

        df[col] = le.fit_transform(
            df[col]
        )

df.head()
```

	Age	MonthlyCharges	Tenure	ContractType	InternetService	CustomerSupportCalls	Churn	
0	56	134	36	0	0	5	0	
1	69	34	36	1	0	5	0	
2	46	138	3	1	1	2	0	
3	32	47	8	1	0	7	0	
4	60	58	70	0	0	0	0	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
X = df.drop(
    "Churn",
    axis=1
)

y = df["Churn"]

print(X.head())
```

```

    Age  MonthlyCharges  Tenure  ContractType  InternetService  \
0    56             134     36             0             0
1    69              34     36             1             0
2    46             138      3             1             1
3    32              47      8             1             0
4    60              58     70             0             0

    CustomerSupportCalls
0                5
1                5
2                2
3                7
4                0
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training:", X_train.shape)
print("Testing :", X_test.shape)
```

```
Training: (800, 6)
Testing : (200, 6)
```

```
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    random_state=42
)

model.fit(
    X_train,
    y_train
)

print("Model Trained")
```

```
Model Trained
```

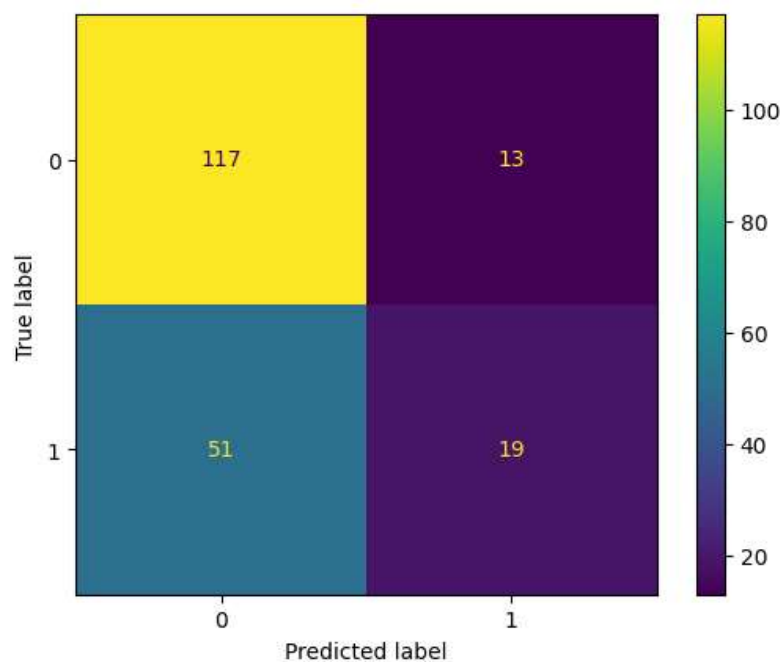
```
y_pred = model.predict(
    X_test
)
```

```
print(y_pred[:10])
```

```
[0 1 0 0 0 0 0 0 0 0]
```

```
cm = confusion_matrix(  
    y_test,  
    y_pred  
)  
  
print(cm)  
  
disp = ConfusionMatrixDisplay(  
    confusion_matrix=cm  
)  
  
disp.plot()  
  
plt.show()
```

```
[[117 13]  
 [ 51 19]]
```



```
accuracy = accuracy_score(  
    y_test,  
    y_pred  
)  
  
precision = precision_score(  
    y_test,  
    y_pred  
)  
  
recall = recall_score(  
    y_test,  
    y_pred  
)  
  
f1 = f1_score(  
    y_test,  
    y_pred  
)  
  
print("Accuracy :", accuracy)  
print("Precision:", precision)
```

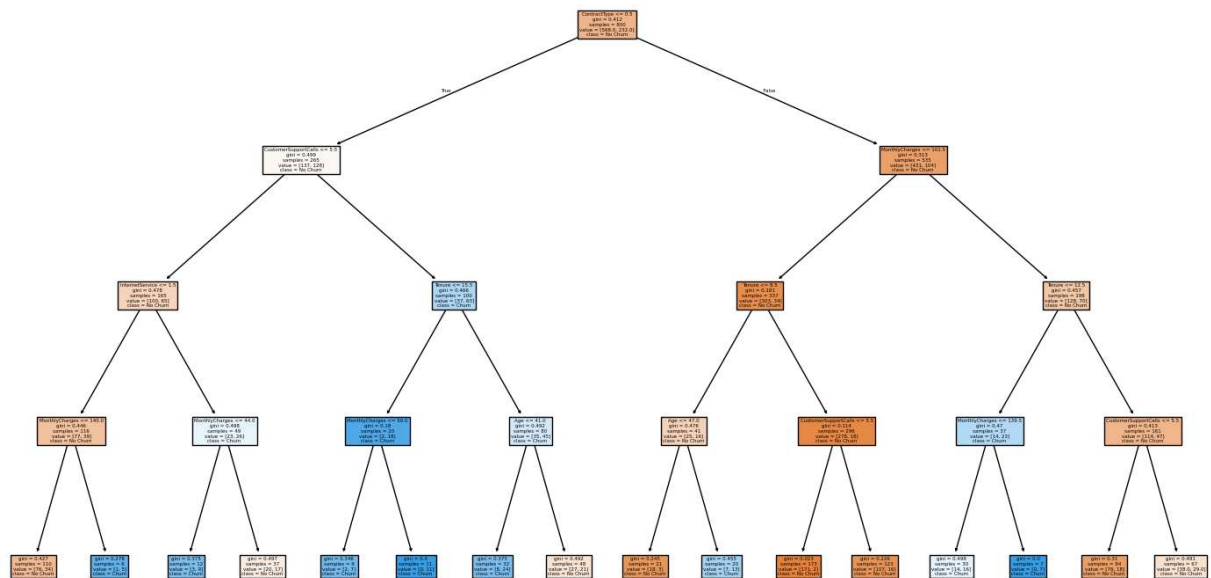
```
print("Recall  :", recall)
print("F1 Score :", f1)
```

```
Accuracy : 0.68
Precision: 0.59375
Recall   : 0.2714285714285714
F1 Score : 0.37254901960784315
```

```
plt.figure(
    figsize=(18,10)
)

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["No Churn", "Churn"],
    filled=True
)

plt.show()
```



```
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

print(importance)
```

	Feature	Importance
3	ContractType	0.306903
1	MonthlyCharges	0.244678
2	Tenure	0.187117
5	CustomerSupportCalls	0.140488
0	Age	0.093749
4	InternetService	0.027065

```
plt.figure(figsize=(8,5))

plt.bar(
```

```

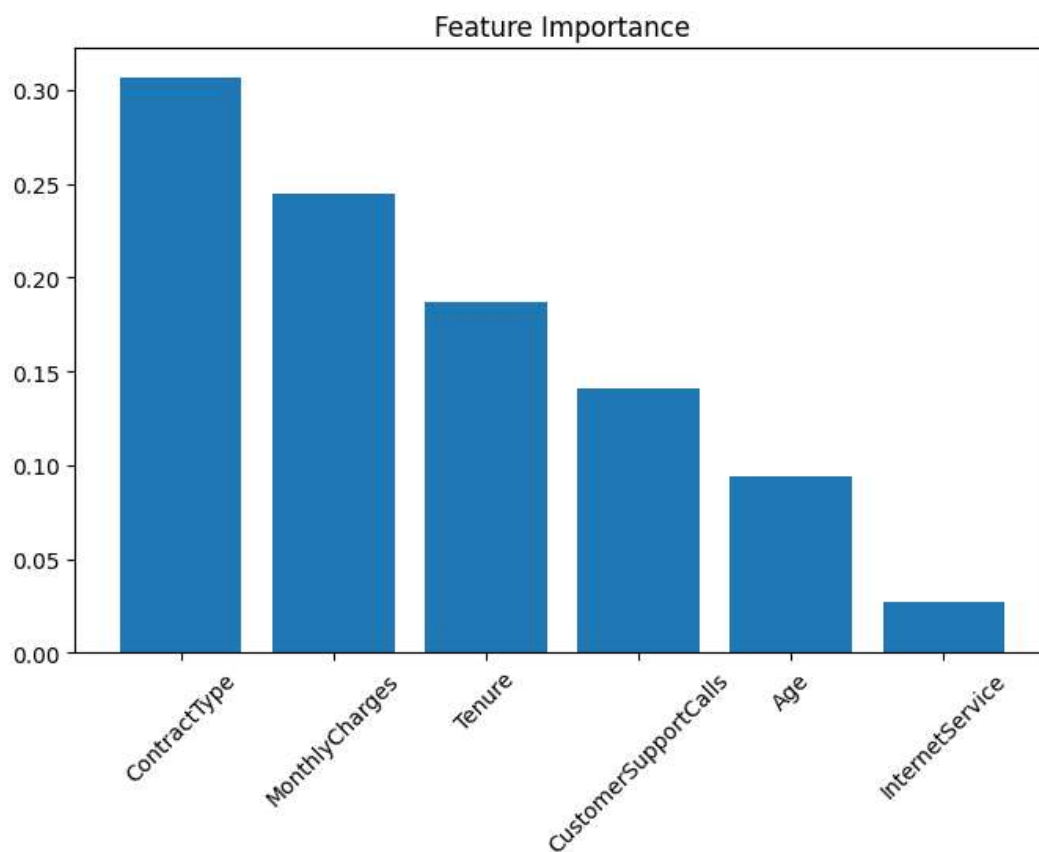
importance["Feature"],
importance["Importance"]
)

plt.xticks(rotation=45)

plt.title(
    "Feature Importance"
)

plt.show()

```



```

print(
    df.groupby("Churn").mean(
        numeric_only=True
    )
)

```

	Age	MonthlyCharges	Tenure	ContractType	InternetService	\
Churn						
0	43.630372	78.661891	37.462751	1.15616	0.992837	
1	44.254967	93.341060	29.728477	0.60596	1.049669	

	CustomerSupportCalls
Churn	
0	4.27937
1	5.36755

```

print("""
BUSINESS RECOMMENDATIONS

1. Focus on month-to-month customers.
2. Improve support quality.
3. Offer long-term contract discounts.
4. Reduce complaints through proactive service.
5. Retain high-value customers using loyalty programs.
""")

```